



Relatório – Implementação das Entidades da Biblioteca *Saber e Cia*



Organização do Projeto

A estrutura do projeto foi reorganizada seguindo boas práticas e separando responsabilidades:

```
/server.js          → Inicia o servidor e importa o app
/src
├── /script
│   └── app.js      → Contém todas as rotas e dados das
entidades
```

Essa organização deixa claro que:

- **server.js** só inicia o servidor.
- **app.js** concentra:
 - dados simulados (arrays)
 - funções de busca
 - rotas para GET/POST/PUT/DELETE



Rotas da API – Biblioteca Saber e Cia

Todas as rotas foram testadas () e funcionam corretamente conforme demonstrado no PDF.

Abaixo estão listadas todas as rotas com base **exatamente no código enviado**.



1. Entidade Livros

✓ GET – Listar todos

GET /livros

✓ GET – Buscar por ISBN

GET /livros/:isbn

✓ POST – Cadastrar livro

POST /livros

✓ PUT – Atualizar livro

PUT /livros/:isbn

✓ DELETE – Remover livro

DELETE /livros/:isbn

Função de busca utilizada:

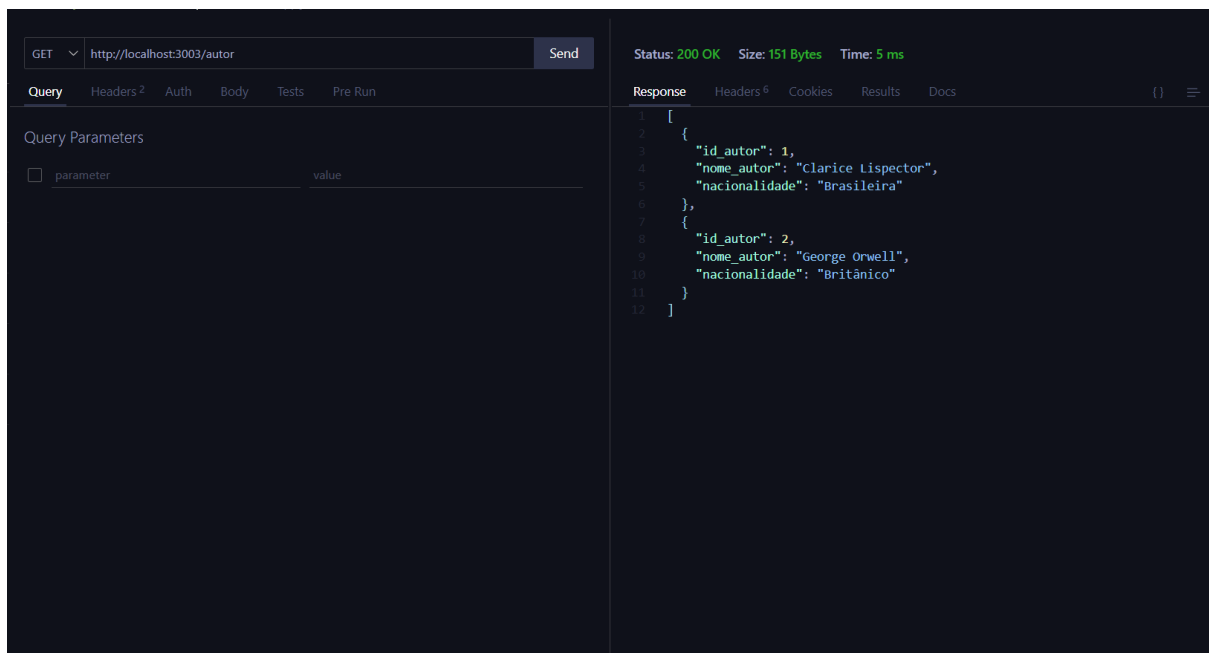
```
function buscarLivro(isbn) {  
  return livros.findIndex((livro) => livro.isbn === Number(isbn));  
}
```



2. Entidade Autor

✓ GET – Listar todos

GET /autor



GET Send

Query Headers² Auth Body Tests Pre Run

Query Parameters

parameter	value
-----------	-------

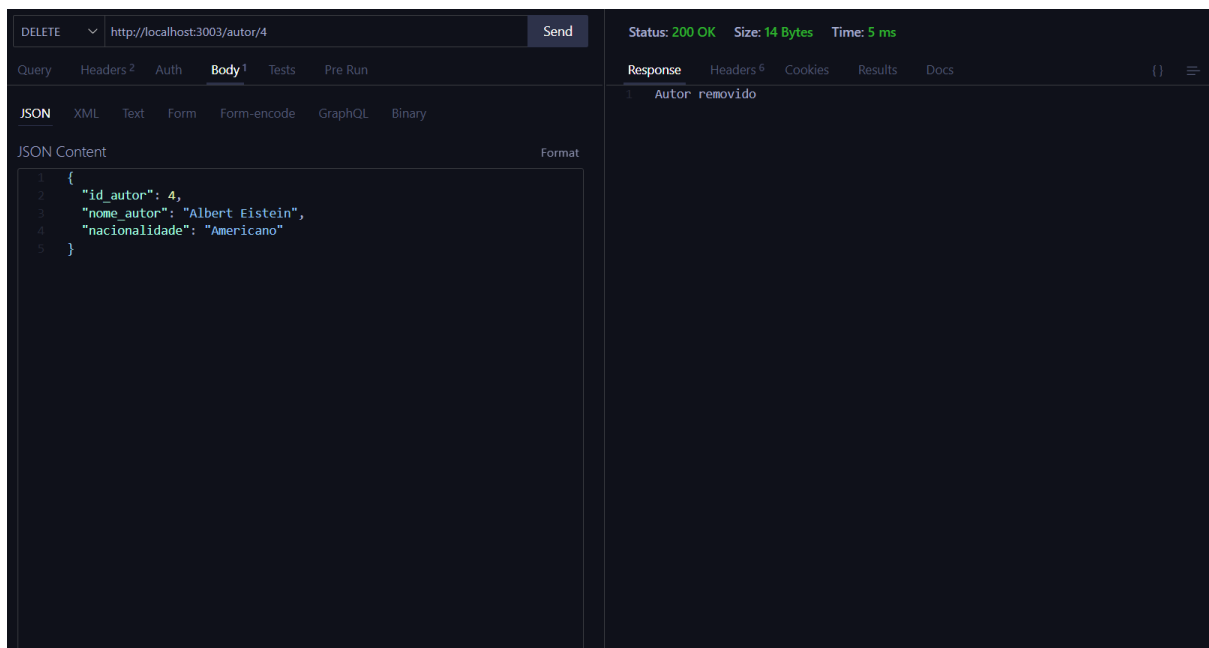
Status: 200 OK Size: 151 Bytes Time: 5 ms

Response Headers⁶ Cookies Results Docs

```
1 [
2   {
3     "id_autor": 1,
4     "nome_autor": "Clarice Lispector",
5     "nacionalidade": "Brasileira"
6   },
7   {
8     "id_autor": 2,
9     "nome_autor": "George Orwell",
10    "nacionalidade": "Britânico"
11  }
12 ]
```

✓ POST – Cadastrar autor

POST /autor



DELETE Send

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id_autor": 4,
3   "nome_autor": "Albert Einstein",
4   "nacionalidade": "Americano"
5 }
```

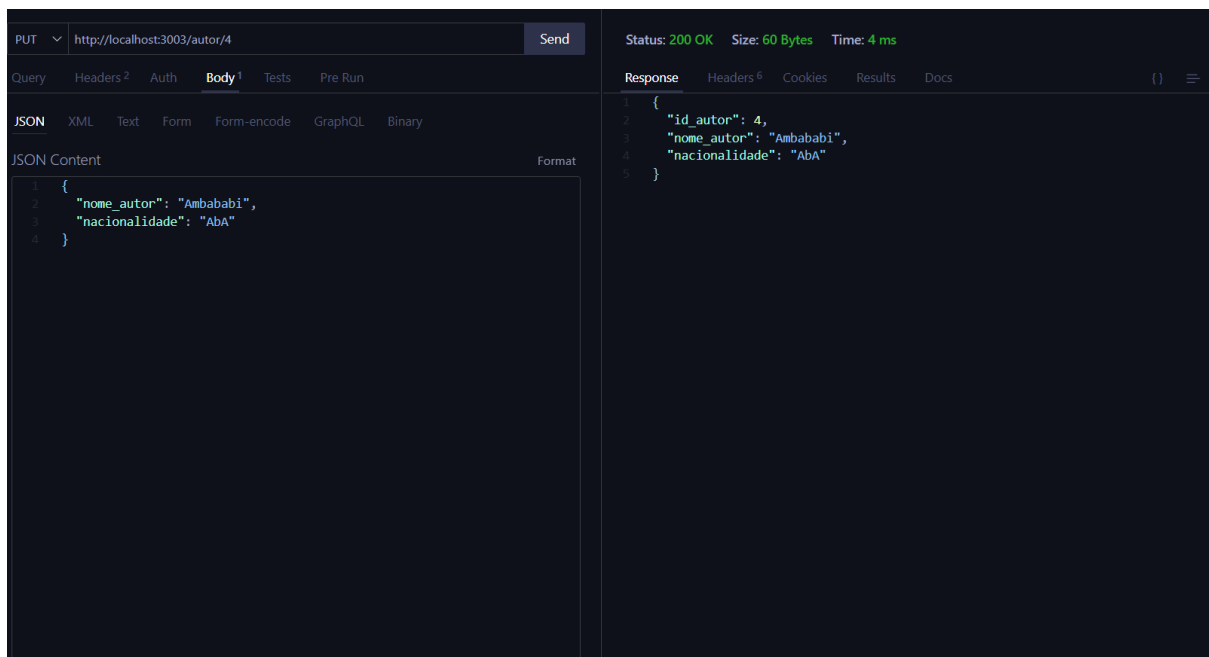
Status: 200 OK Size: 14 Bytes Time: 5 ms

Response Headers⁶ Cookies Results Docs

```
1 Autor removido
```

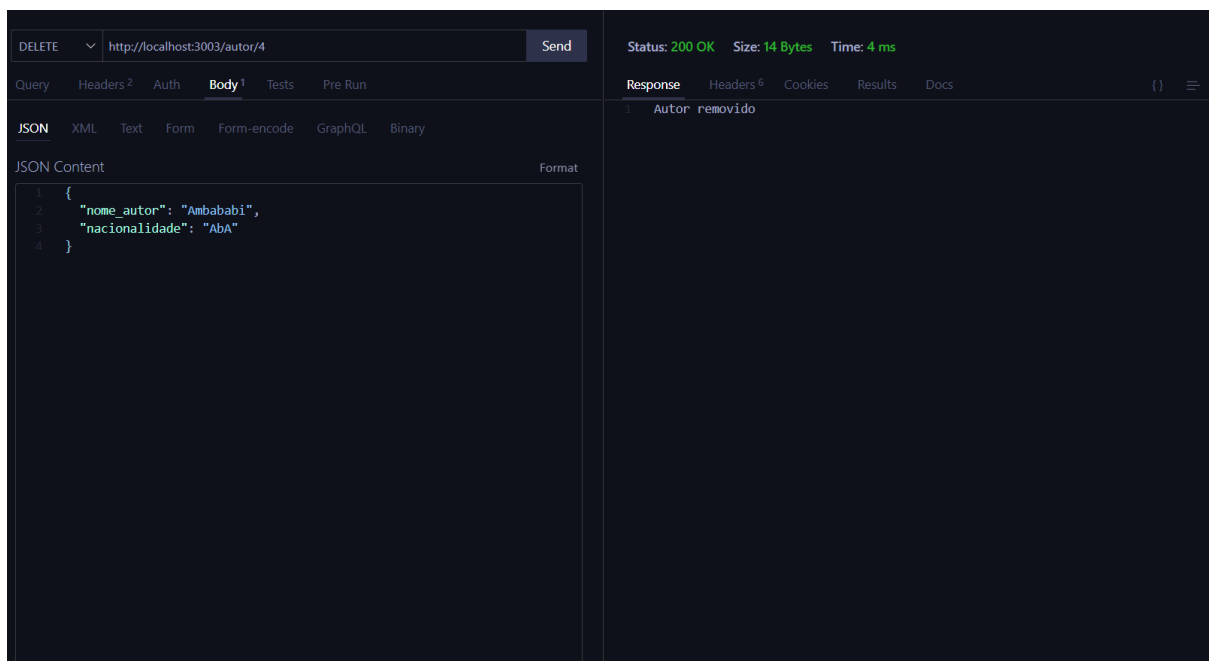
✓ PUT – Atualizar autor

PUT /autor/:id_autor



✓ DELETE – Remover autor

DELETE `/autor/:id_autor`



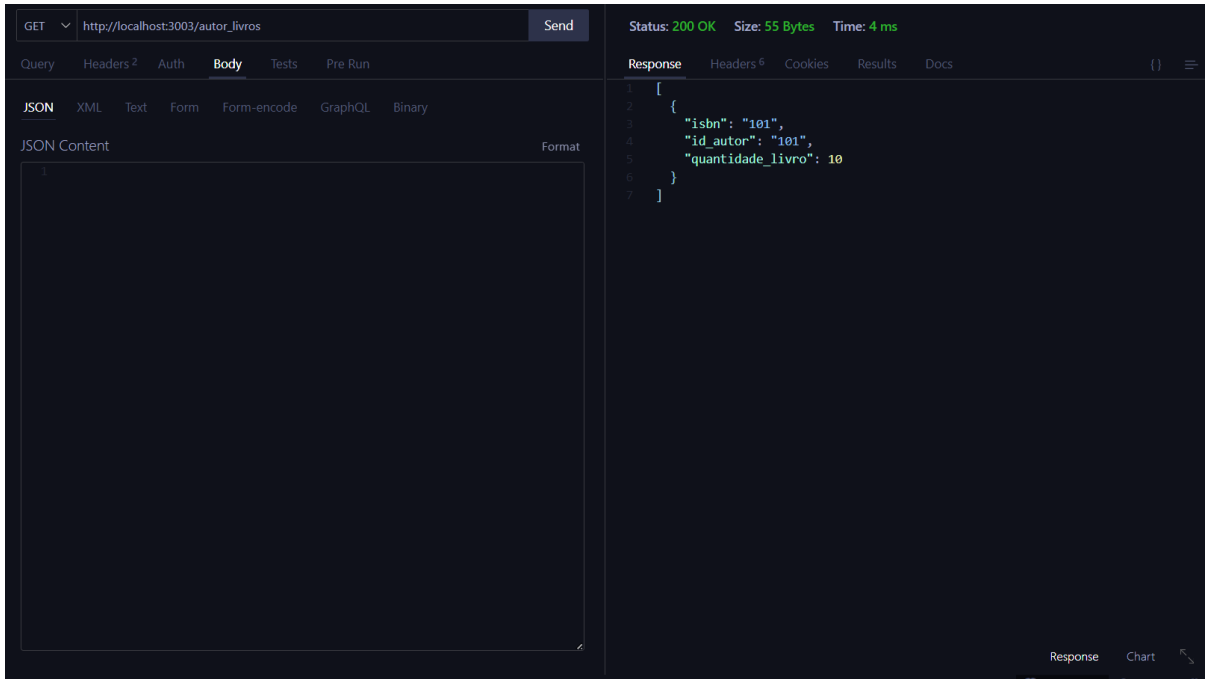
Função de busca:

```
function buscarAutor(id_autor) {
  return autor.findIndex((auto) => auto.id_autor === Number(id_autor));
}
```

3. Entidade Autor_Livro (Relação N:N)

✓ GET – Listar todos

GET /autor_livros



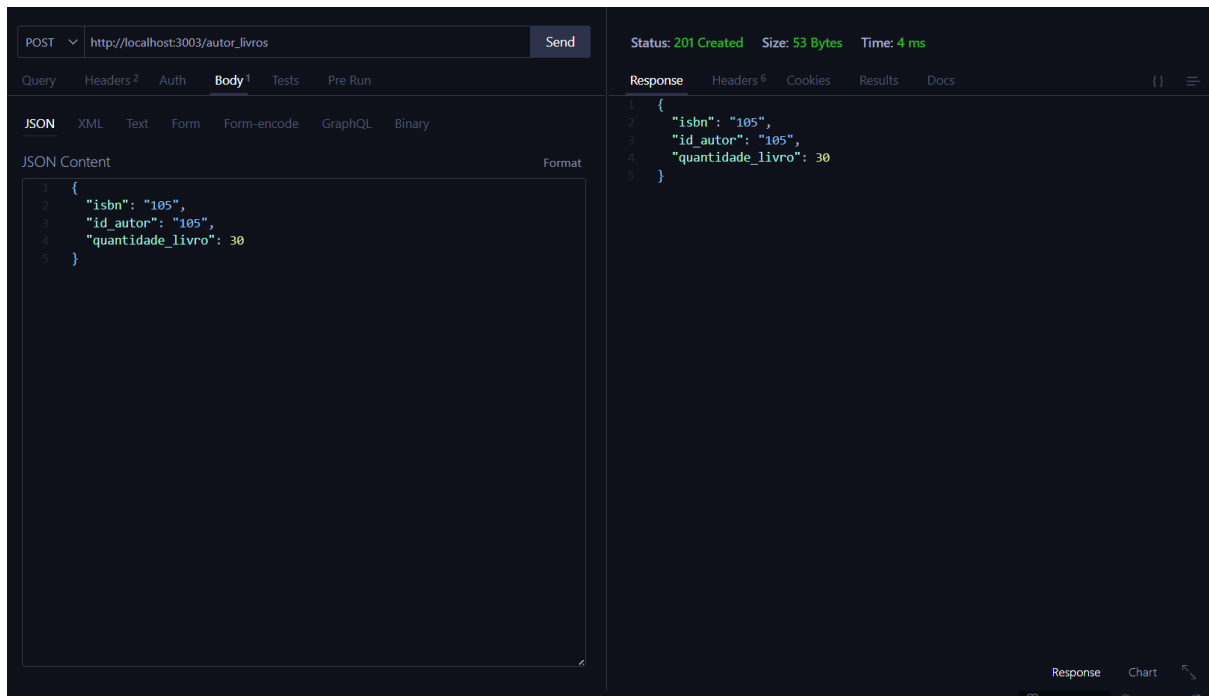
The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3003/autor_livros
- Status:** 200 OK
- Size:** 55 Bytes
- Time:** 4 ms
- Response Body (JSON):**

```
[
  {
    "isbn": "101",
    "id_autor": "101",
    "quantidade_livro": 10
  }
]
```

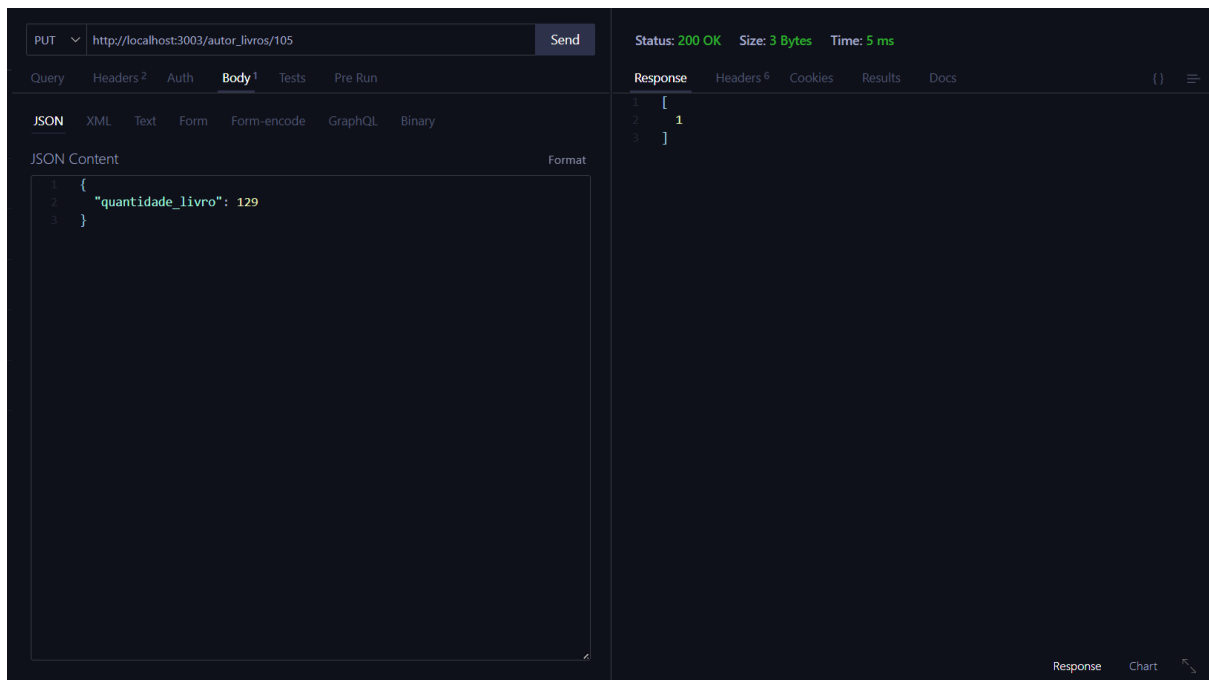
✓ POST – Cadastrar relação

POST /autor_livros



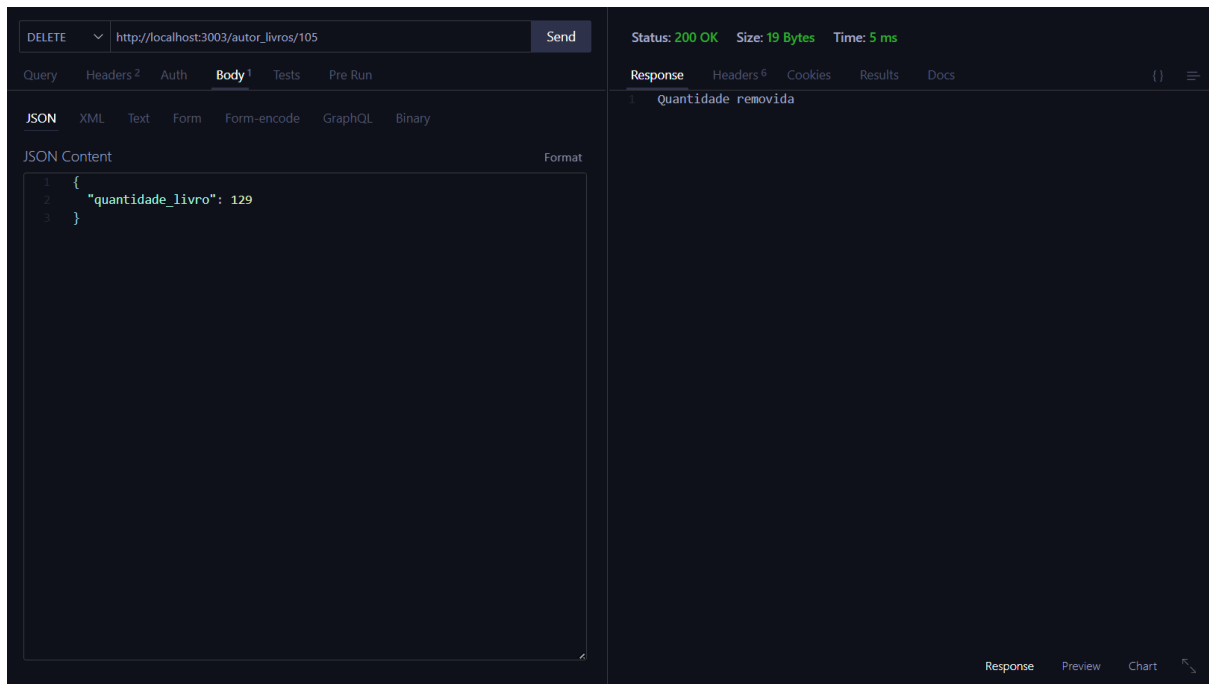
✓ PUT – Editar quantidade

PUT /autor_livros/:isbn



✓ DELETE – Excluir registro

DELETE /autor_livros/:isbn



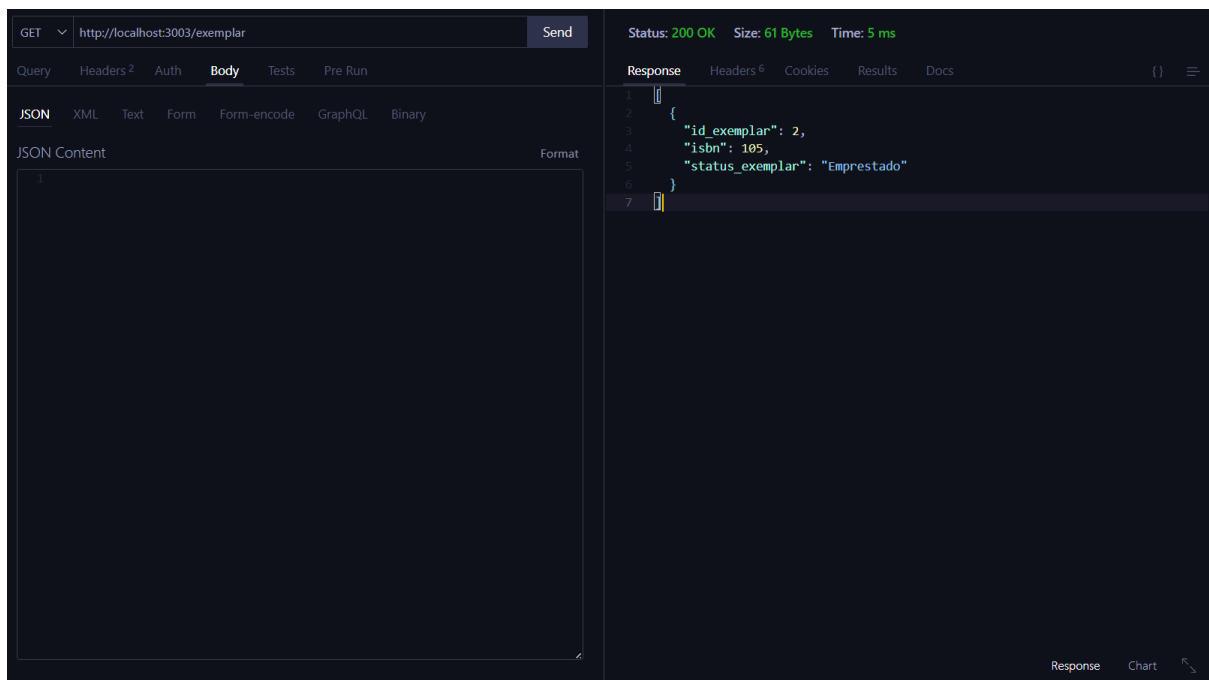
Função de busca:

```
function buscarAutor_livro(isbn) {  
  return autor_livros.findIndex((autor_livro) => autor_livro.isbn  
    ===Number(isbn));  
}
```

4. Entidade Exemplar

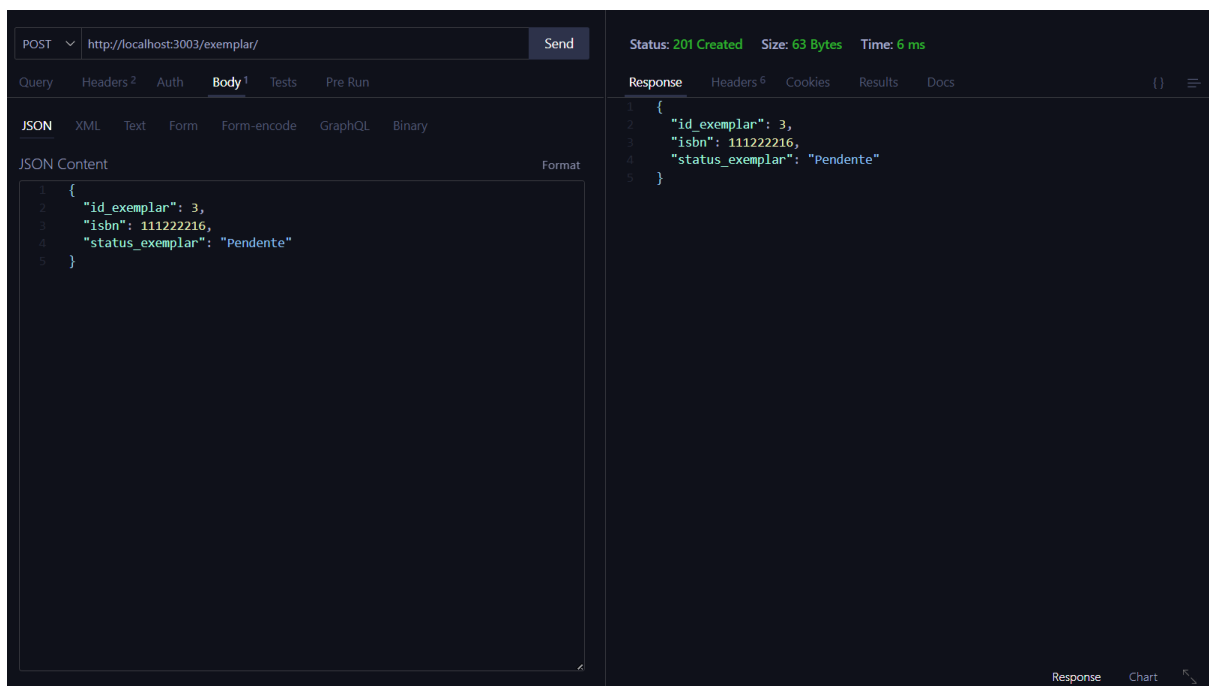
✓ GET – Listar

GET /exemplar



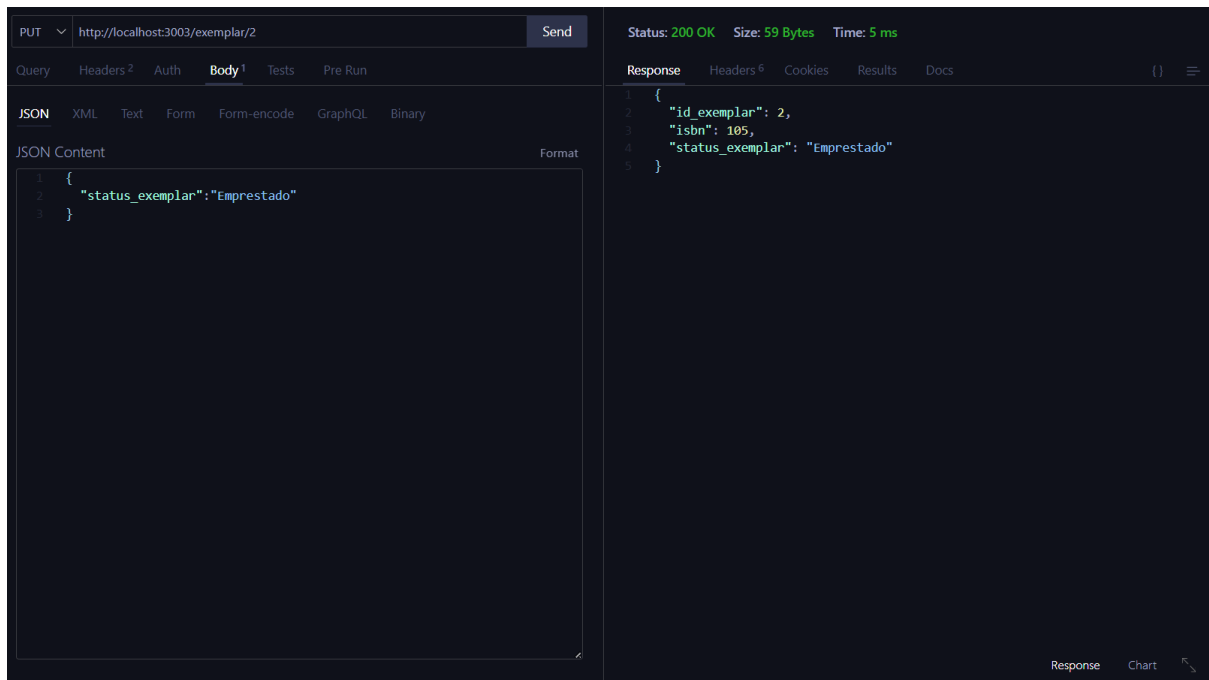
✓ POST – Cadastrar exemplar

POST /exemplar



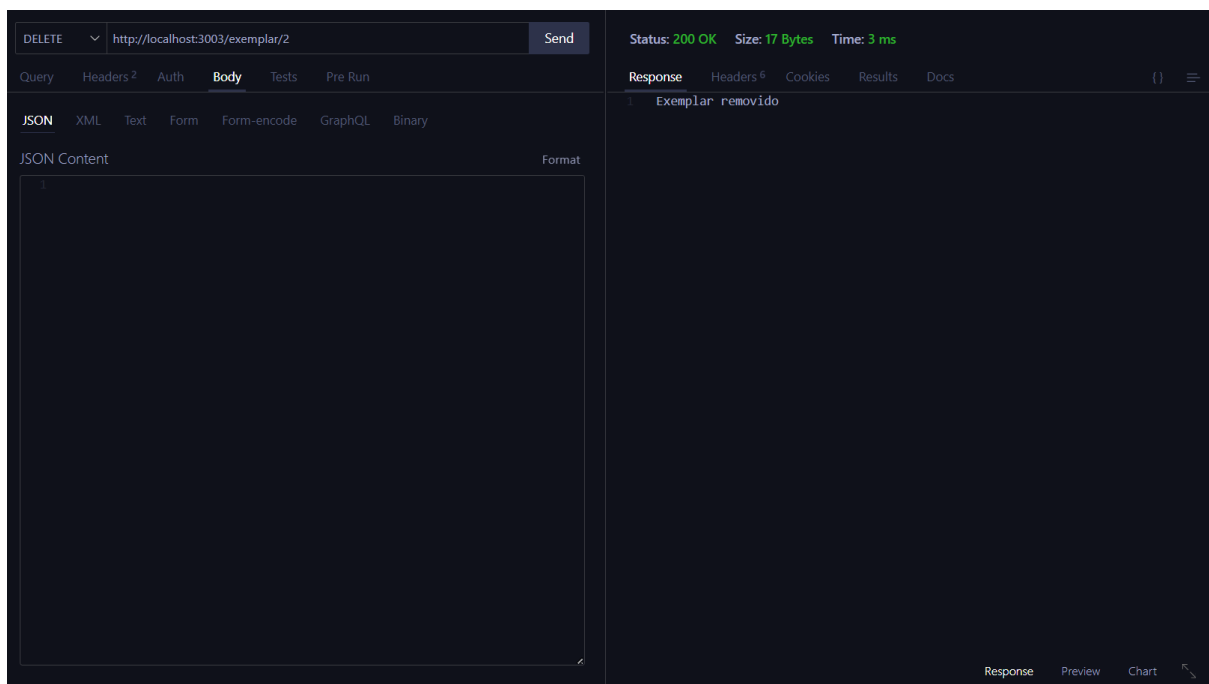
✓ PUT – Atualizar status

PUT /exemplar/:id_exemplar



✓ DELETE – Remover exemplar

DELETE `/exemplar/:id_exemplar`



Função de busca:

```
function buscarExemplar(id_exemplar) {
  return exemplar.findIndex((exemplars) => exemplars.id_exemplar ===
Number(id_exemplar));
}
```



5. Entidade Membros

✓ GET – Listar

GET /membros

The screenshot shows a REST client interface with a GET request to `http://localhost:3003/membros` and its response. The response is a JSON array containing one object with the following fields:

```
[
  {
    "matricula_aluno": 1125002212,
    "nome_completo": "Jonas Daniel de Brito Lopes",
    "endereco_aluno": "É lal",
    "telefone_contato": "11 93301-2244"
  }
]
```

✓ POST – Cadastrar membro

POST /membros

POST `http://localhost:3003/membros/` Send

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "matricula_aluno": 1125004334,
3   "nome_completo": "ASDADS",
4   "endereco_aluno": "BAHIA",
5   "telefone_contato": "789080808080"
6 }
```

Status: 201 Created Size: 114 Bytes Time: 5 ms

Response Headers⁶ Cookies Results Docs {} ≡

```
1 {
2   "matricula_aluno": 1125004334,
3   "nome_completo": "ASDADS",
4   "endereco_aluno": "BAHIA",
5   "telefone_contato": "789080808080"
6 }
```

Response Chart ↗

✓ PUT – Atualizar membro

PUT `/membros/:matricula_aluno`

PUT `http://localhost:3003/membros/1125004334` Send

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "nome_completo": "ASDADS@@#@!ASDASD",
3   "endereco_aluno": "BAHIA",
4   "telefone_contato": "789080808080"
5 }
```

Status: 200 OK Size: 125 Bytes Time: 6 ms

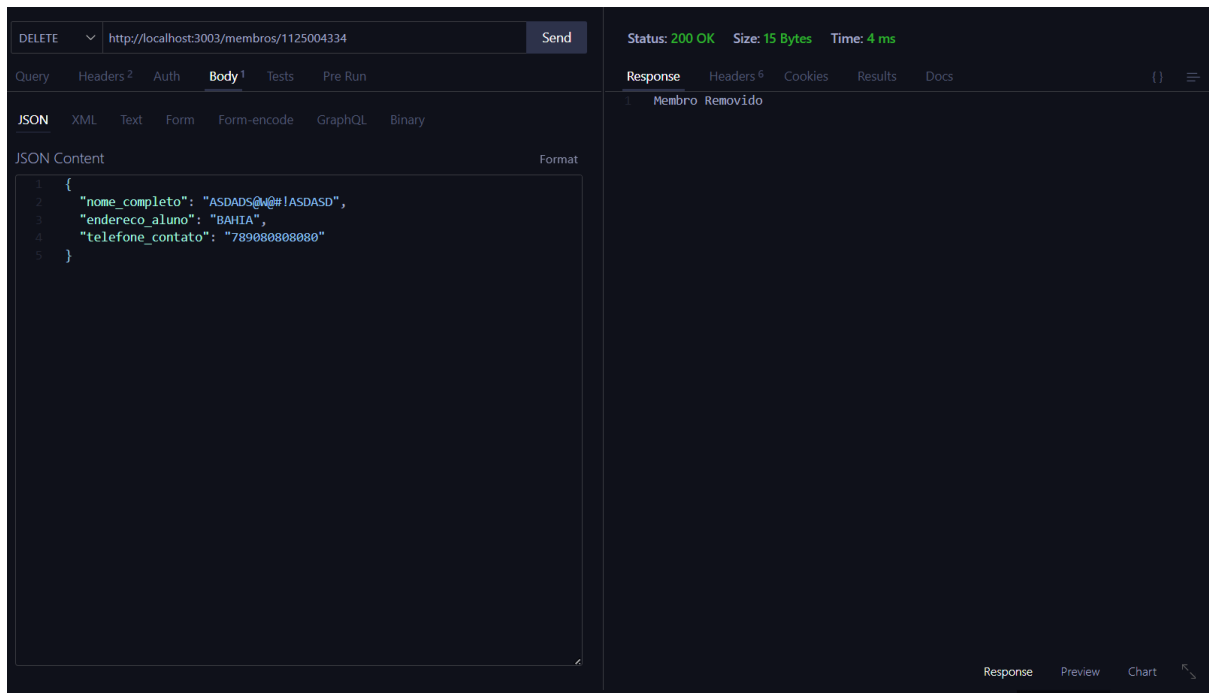
Response Headers⁶ Cookies Results Docs {} ≡

```
1 {
2   "matricula_aluno": 1125004334,
3   "nome_completo": "ASDADS@@#@!ASDASD",
4   "endereco_aluno": "BAHIA",
5   "telefone_contato": "789080808080"
6 }
```

Response Chart ↗

✓ DELETE – Remover membro

DELETE `/membros/:matricula_aluno`



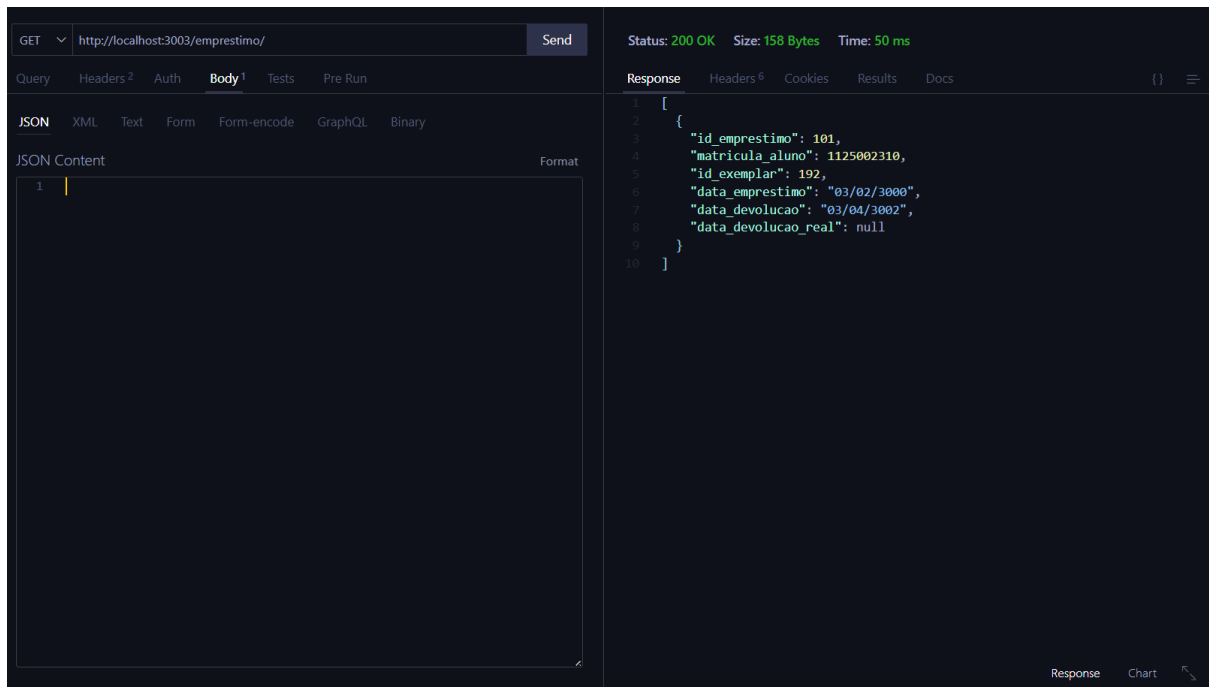
Função de busca:

```
function buscarMembros(matricula_aluno) {  
  return membros.findIndex((membro) => membro.matricula_aluno ===  
    Number(matricula_aluno));  
}
```

6. Entidade Empréstimo

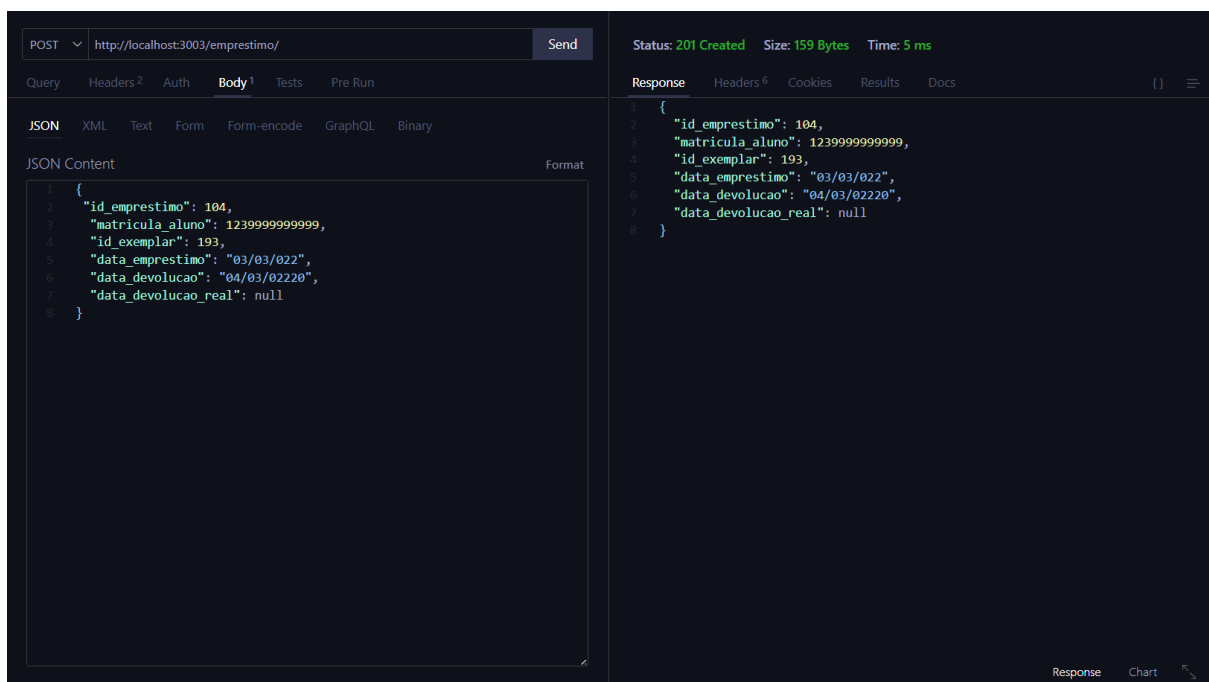
✓ GET – Listar empréstimos

GET /emprestimo



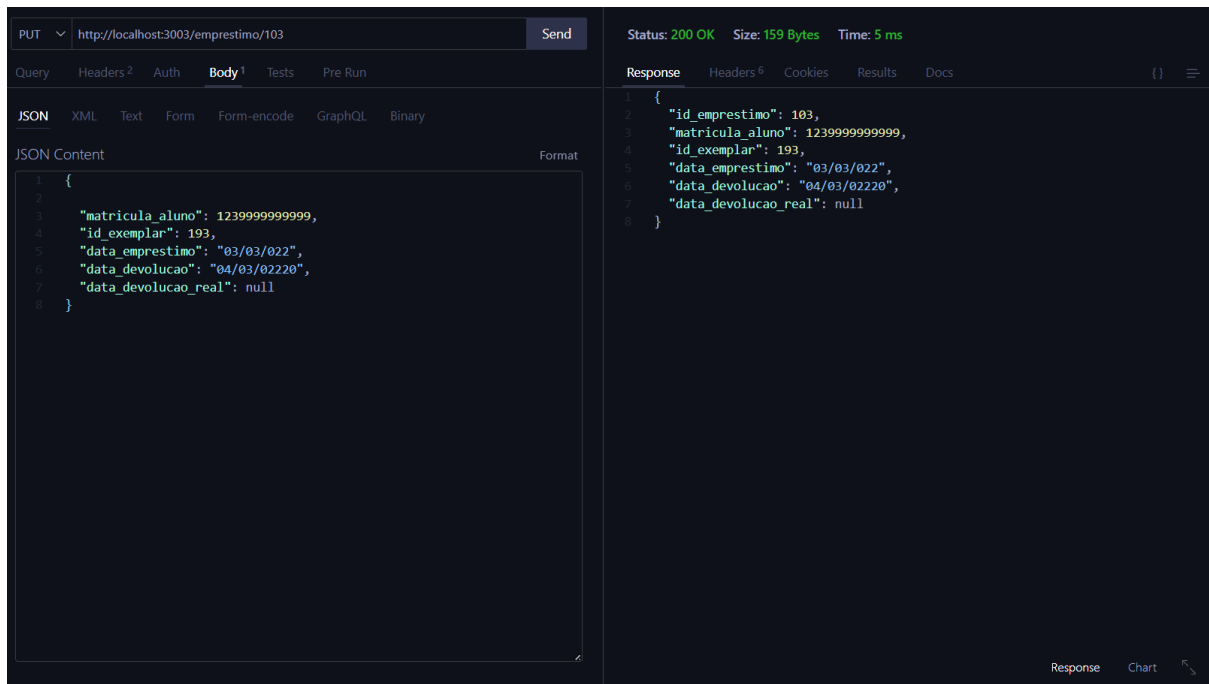
✓ POST – Cadastrar empréstimo

POST `/emprestimo`



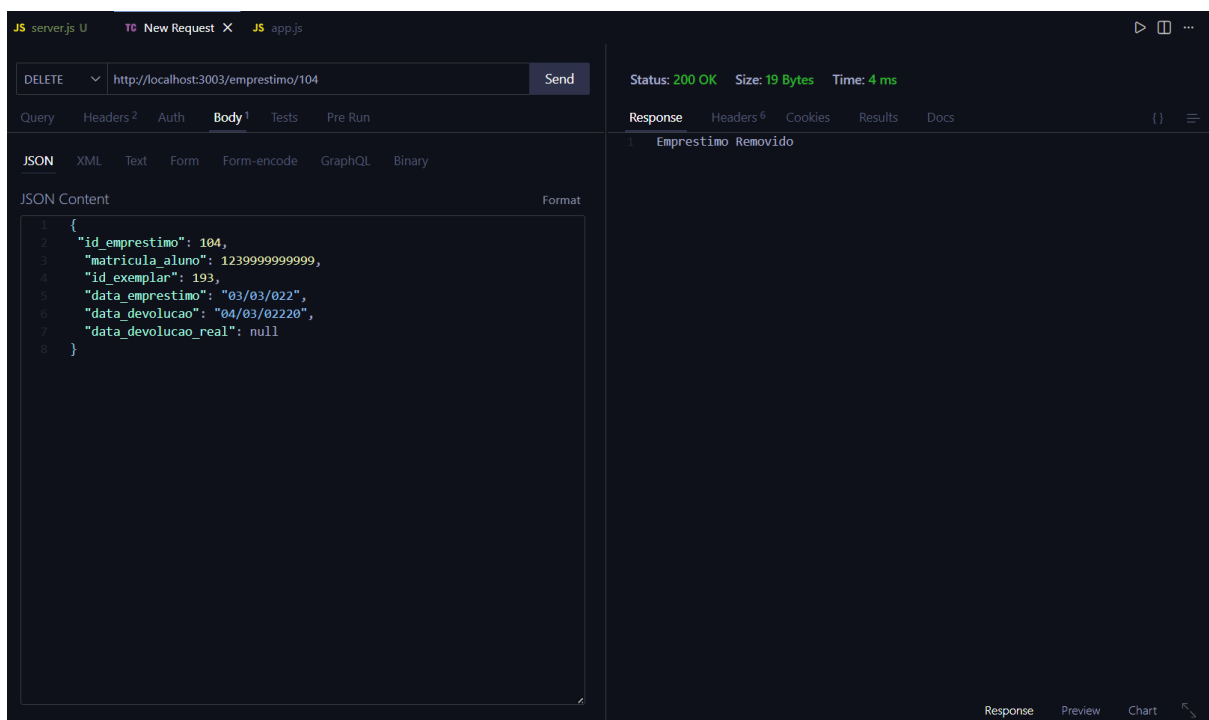
✓ PUT – Atualizar dados

PUT `/emprestimo/:id_emprestimo`



✓ DELETE – Remover empréstimo

DELETE `/emprestimo/:id_emprestimo`



Função de busca:

```
function buscarEmprestimo(id_emprestimo) {
  return emprestimo.findIndex((emprest) => emprest.id_emprestimo ===
    Number(id_emprestimo));
}
```

}



Ferramentas Utilizadas

- Node.js
- Express.js
- REST API
- Thunder Client / Postman (para testes)
- VS Code



Dificuldades Enfrentadas

- Descobrir a fonte do erro dado na aplicação.
- Resolver o relacionamento N:N na entidade Autor-Livro.
- Escrever repetidas vezes o mesmo código, alterando alguns parâmetros.